

Module 1

INTRODUCTION

UNIX COMPONENTS/ARCHITECTURE:

The kernel is the core of the operating system. It is a collection of routines written in C. It is loaded into memory when the system is booted and communicates directly with the hardware. User programs that need to access the hardware use the services of the kernel via use of system calls and the kernel performs the job on behalf of the user. Kernel is also responsible for managing system's memory, schedules processes, decides their priorities.

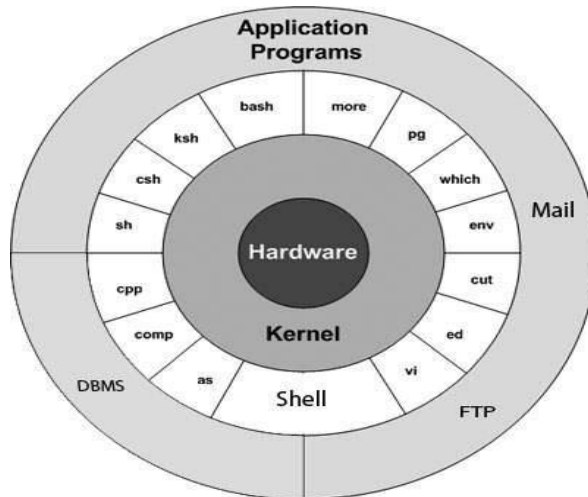


Figure 1.1 Unix Architecture

Kernel: is the core of operating system. A collection of routines mostly written in C. It is loaded into memory when the system is booted and communicates directly with the hardware. The kernel manages system memory, processes, decides priorities.

Shell: interface between Kernel and User. It functions as **command interpreter** i.e it receives and interprets the command from user and interacts with the hardware. There is only one kernel running on the system, there could be several shells in action- one for each user who is logged in.

Files and Process: file is an array of bytes and it contain virtually anything. Unix considers even the directories and devices as members of file system. The dominant file type is text and behavior of system is mainly controlled by text files.

The second entity is the process, which is the name given to a file when it is executed as a program. Process is simply a time image of an executable file.

System Calls: Though there are thousands of commands in the unix system, they all use a handful of functions called system calls. User programs that need to access the hardware use the services of the kernel, which performs the job on users behalf. These programs access the kernel through a set of functions called system calls.

Ex: open()-- system call to access both file and device. Write()—system call to write a file.

FEATURES OF UNIX

Several features of UNIX have made it popular. Some of them are:

Portable: UNIX can be installed on many hardware platforms. Its widespread use can be

traced to the decision to develop it using the C language. Because C programs are easily moved from one hardware environment to another, it is relatively simple to port it to different environments.

Multiuser: The UNIX design allows multiple users to concurrently share hardware and software

Multitasking: UNIX allows a user to run more than one program at a time. In fact more than one program can be running in the background while a user is working foreground.

Networking: While UNIX was developed to be an interactive, multiuser, multitasking system, networking is also incorporated into the heart of the operating system. Access to another system uses a standard communications protocol known as Transmission Control Protocol/Internet Protocol (TCP/IP).

Organized File System: UNIX has a very organized file and directory system that allows users to organize and maintain files.

Device Independence: UNIX treats input/output devices like ordinary files. Input or output to a program can be from any device or file. The source or destination for file input and output is easily controlled through a UNIX design feature called redirection.

Utilities: UNIX provides a rich library of utilities that can be use to increase user productivity.

Services: UNIX also includes the support utilities for system administration and control.

THE UNIX ENVIRONMENTS AND UNIX STRUCTURE

There 3 different environments in UNIX

- Personal environment
- Timesharing environment: Many users connected to one computer
- Client/server environment Computing split between a central computer (server) and users' computers (clients)

Personal environment originally unix designed as a multiuser environment, many user user are installed UNIX on their personal computers this tends to personal unix system environment

Timesharing environment

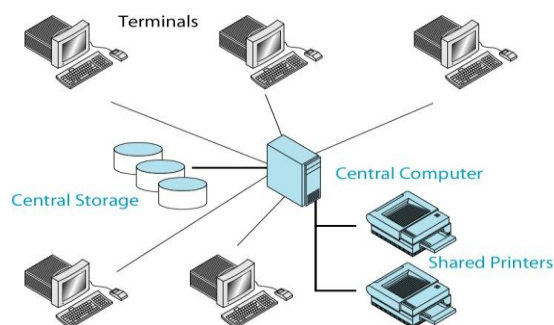


Figure 1.3.1 Time sharing machine

In a time sharing environment, many users are connected to one or more computers. The output devices (such as printers) and auxiliary storage devices (such as disks) are shared by

all users .In this environment all of the computing must be done by the central computer.

The central computer has several responsibilities

It must control the shared resources. It must manage the shared data

It must manage the printing data and resource. It must handle the computing all task

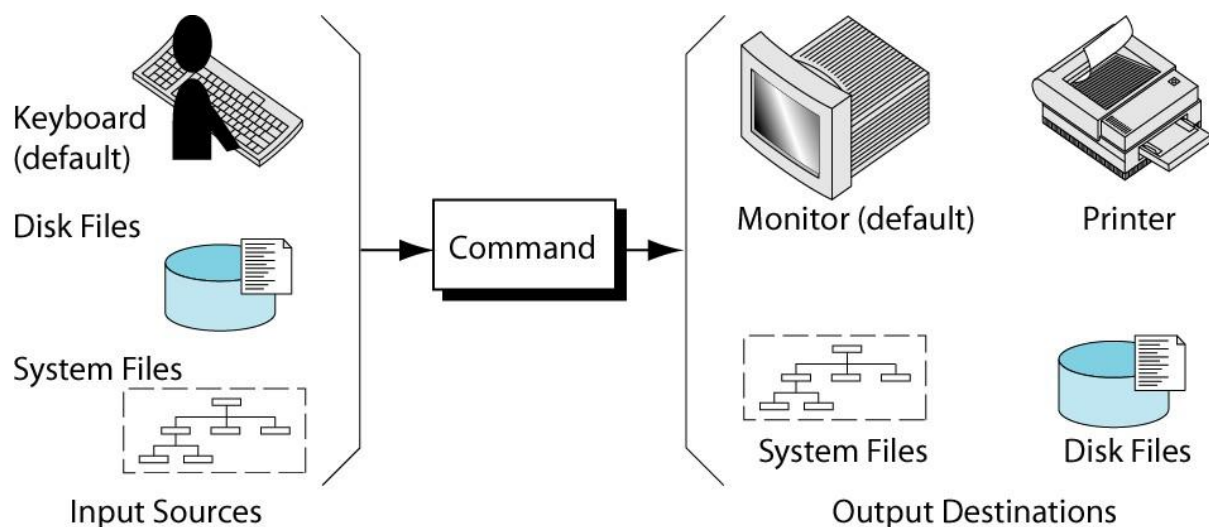
All of this work tends to keep the central computer busy so, user has to wait more time for get done their work so, it is non-productive because of slow response.

POSIX AND SINGLE UNIX SPECIFICATION

Dennis Ritchie's decision to rewrite UNIX in C didn't quite make UNIX very portable. UNIX fragmentation and absence of a single conforming standard adversely affected the development of portable applications. First ,AT &T created the System V Interface Definition(SVID). Later, X/Guide(XPG). Products conforming to this specification were branded UNIX95, UNIX98 or UNIX03 depending on the version of the specification. Yet another group of standards, the portable operating system interface for computer environments(POSIX), were developed at the behest of the Institution of Electrical and Electronics Engineers(IEEE). POSIX refers to operating systems in general, but was based on UNIX. Two of the most cited standards from the POSIX family are known as POSIX.1 and POSIX.2. POSIX.1 specifies the C application program interface the system calls. POSIX.2 deals with the shell and utilities.

In 2001, a joint initiative of X/Open and IEEE resulted in the unification of the two standards. This is the single UNIX Specification, version 3(SUSV3). The “write once, adopt everywhere” approach to this development means that once software has been developed on any POSIX compliant UNIX system, it can be easily ported to another POSIX- compliant UNIX machine with minimum modifications. We make reference to POSIX throughout this text, but these references should be interpreted to mean the SUSV3 as well.

GENERAL FEATURES OF UNIX COMMANDS/ COMMAND STRUCTURE



Features of command structure

Commands are entered at shell prompt. The components of the command line are: the verb;

□

any options required by the command the command's arguments (if required).

For example, the general form of a UNIX command is:

\$verb [-option(s)] [argument(s)]

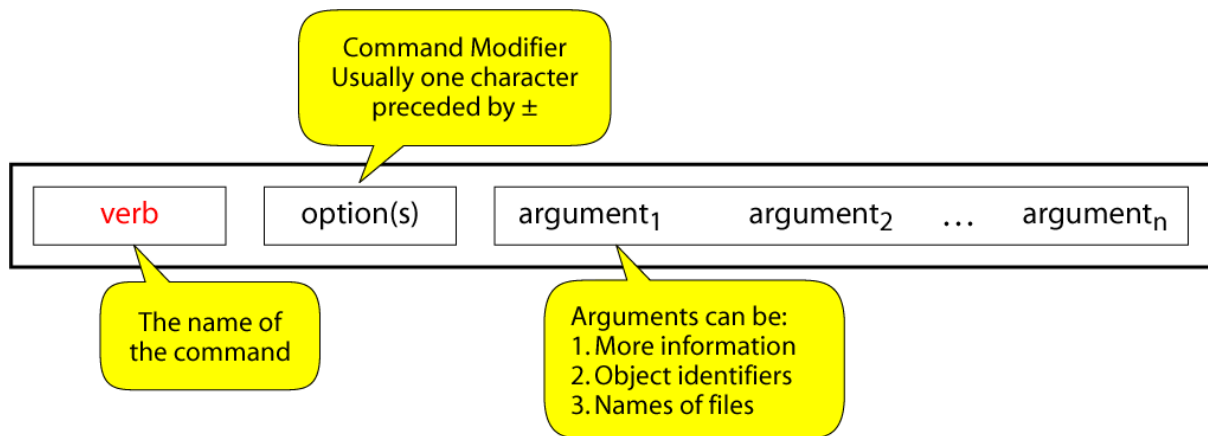


Figure 1.5.2 command shell

Verb: is the command name. The command indicates what action is to be taken. This action concept gives us the name verb for action.

Option: modifies how the action is applied.

Argument: provides additional information to the command.

Note: Options **MUST** come after the command and before any command arguments. Options **SHOULD NOT** appear after the main argument(s). However, some options can have their own arguments if **options** are enclosed within the `[]` then options are not mandatory else it is compulsory if **arguments** are enclosed within the `[]` then options are not mandatory else it is compulsory.

BASIC COMMANDS SUCH AS `echo`, `printf`, `ls`, `who`, `date`, `passwd`, `cal`

date: displays the system date and time. If the system is local that is one in your own area-it is the current time. If the system is remote, such as across the country the reply will contain the time where the system is physically located.

The input for the date is the system itself. The date is actually maintained in the computer as a part of OS. The date command sends its response to monitor.

\$date

Sun Aug 28 13:28:39 IST 2016

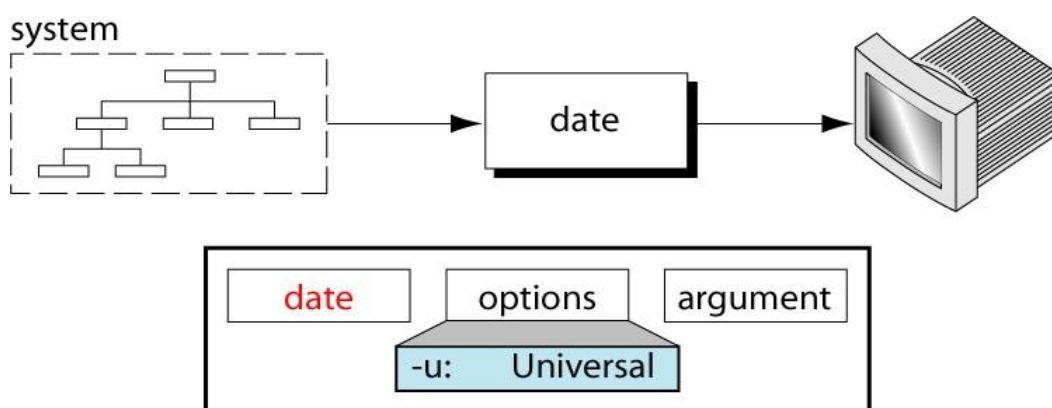


Figure 1.6.1 system date

```
$date -u
```

```
Sun Aug 28 08:02:13 UTC 2016
```

```
$ date "+Today's date is %D and Time is %T"
```

```
Today's date is 08/28/16 and Time is 13:33:57
```

cal: The Calendar Command

system

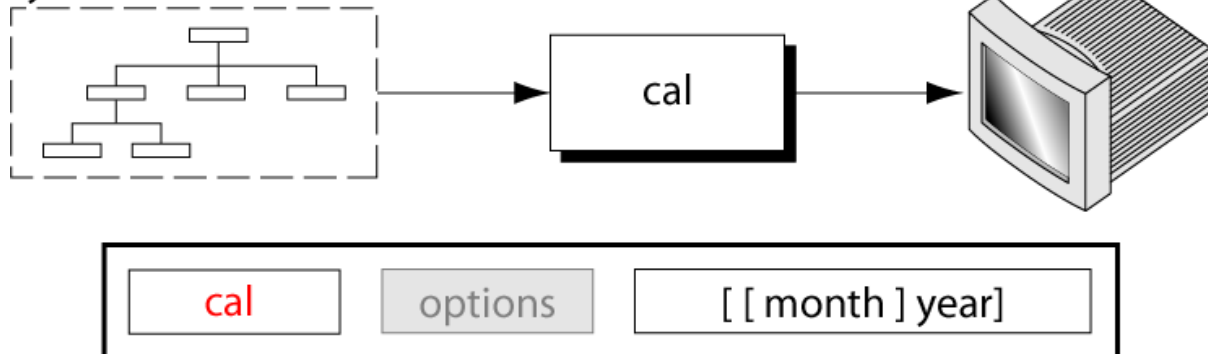


Figure 1.6.2 calendar command

A single parameter specifies the 4 digit year (1 - 9999) to be displayed. Two parameters denote the Month (1 - 12) and Year (1 - 9999). If arguments are not specified, the current month is displayed. A year starts on 01 Jan.

To display current month's calendar

```
$ cal
```

```
April 2016
```

```
Su Mo Tu We Th Fr Sa 1 2
```

```
3 4 5 6 7 8 9
```

```
10 11 12 13 14 15 16
```

```
17 18 19 20 21 22 23
```

```
24 25 26 27 28 29 30
```

To display feb 2015 calendar

```
$ cal 2 2015
```

```
February 2015
```

```
Su Mo Tu We Th Fr Sa 1 2 3 4 5 6 7
```

```
8 9 10 11 12 13 14
```

```
15 16 17 18 19 20 21
```

```
22 23 24 25 26 27 28
```

To display complete year calendar.

```
$ cal -y
```

who: The Who is online Command

The **who** command prints information about all users who are currently logged in.

system

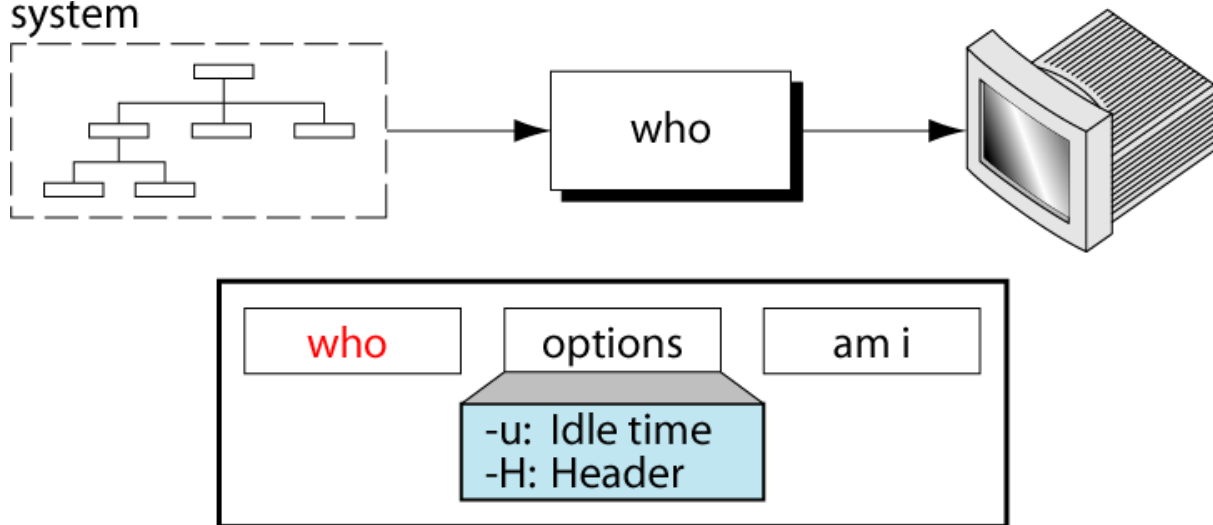


Figure 1.6.3 who command

who [OPTION]... [FILE] [am i]

-u –Idle time: Print the idle time for each user, and the process ID.

-H – HEADING : Print a line of column headings.

\$who

Displays the username, terminal, and time and date of all currently logged-in sessions

```
vizion ~ tty7 2016-08-28 13:28 (:0)
vizion pts/0 2016-08-28 13:28 (:0.0)
```

\$who am i

Displays the same information, but only for the terminal session where the command was issued, for example:

```
vizion pts/0 2016-08-28 13:28 (:0.0)
```

\$who -u

Indicates how long it has been since there was any activity on the line. This is known as **Idle time**. It also returns the process id for the user.

```
vizion tty7 2016-08-28 13:28 . 2420 (:0)
vizion pts/0 2016-08-28 13:28 . 2623 (:0.0)
```

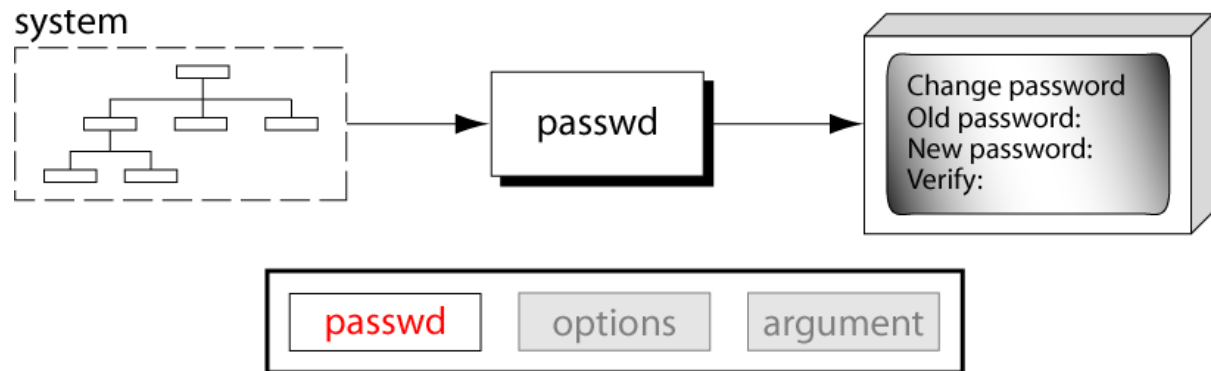
\$who -uH

Displays "all" information, and headers above each column of data, for example:

NAME	LINE	TIME	IDLE	PID	COMMENT	EXIT
		2016-08-28 13:26		1262	id=l5	term=0 exit=
0						
LOGIN	tty1	2016-08-28 13:26		2303	id=1	
LOGIN	tty2	2016-08-28 13:26		2304	id=2	
LOGIN	tty3	2016-08-28 13:26		2305	id=3	
LOGIN	tty4	2016-08-28 13:26		2306	id=4	
LOGIN	tty5	2016-08-28 13:26		2307	id=5	
LOGIN	tty6	2016-08-28 13:26		2308	id=6	
		2016-08-28 13:26		2309	id=x	
vizion	+ tty7	2016-08-28 13:28	.	2420	(:0)	
vizion	+ pts/0	2016-08-28 13:28	.	2623	(:0.0)	

passwd command: The change Password(PASSWD) COMMAND

The **passwd** command is used to change the password of a user account. A normal user can run **passwd** to change their own password, and a system administrator (the superuser) can use **passwd** to change another user's password, or define how that account's password can be used or changed.



password command

passwd syntax

passwd [*OPTION*] [*USER*]

\$passwd

Running **passwd** with no options will change the password of the account running the command. You will first be prompted to enter the account's current password: (current) UNIX password:

If it is correct, you will then be asked to enter a new password: Enter new UNIX password:

and to enter the same password again, to verify it: Retype new UNIX password:

If the passwords match, the password will be changed.

The **passwd** command changes passwords for user accounts. A normal user can only change the password for their own account, but the superuser can change the password for any account. **Passwd** can also change or reset the account's validity period — how much time can pass before the password expires and must be changed. Before a normal user can change their own password, they must first enter their current password for verification. (The superuser can bypass this step when changing another user's password.)

After the current password has been verified, **passwd** checks to see if the user is allowed to change their password at this time. If not, **passwd** refuses to continue, and exits. Otherwise, the user is then prompted twice for a replacement password. Both entries must match for **passwd** to continue.

Rules for giving passwords

must be ≥ 6 characters long,

must contain 2 out of 3 of upper-case letters,

lower-case letters,

non-letters (digits, punct)

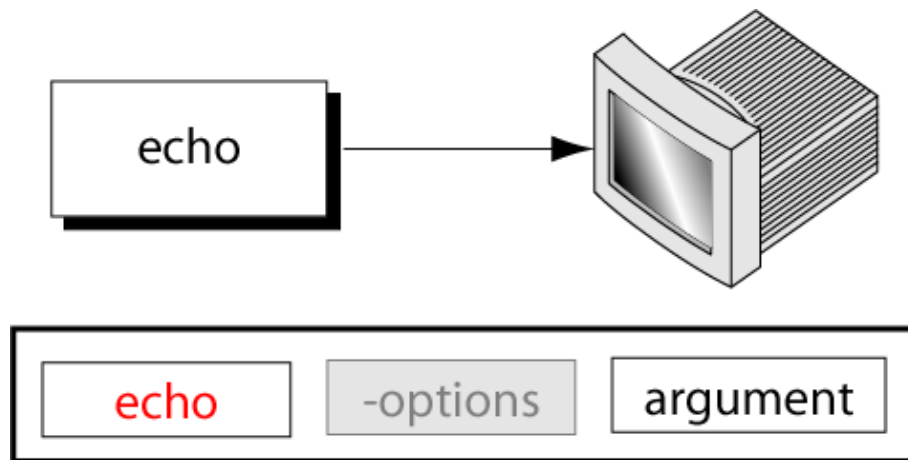
may not be a dictionary word or too similar to your name

echo: Print message command

The **echo** command used to display a message.

To display the diagnostic messages on the terminal or to issue prompts for taking user input

To evaluate shell variable.



Echo command

\$ echo [SHORT-OPTION]... [STRING]...

Options

-n Do not output a trailing newline.

-e Enable interpretation of backslash escape sequences (see below for a list of these).

-E Disable interpretation of backslash escape sequences (this is the default).

--help Display a help message and exit.

version Output version information and exit.

If you specify the **-e** option, the following escape sequences are recognized:

**** A literal backslash character ("****").

\a An alert (The BELL character).

\b Backspace.

\c Produce no further output after this.

\e The escape character; equivalent to pressing the escape key.

\f A form feed.

\n A newline.

\r A carriage return.

\t A horizontal tab.

\v A vertical tab.

Ex 1: **\$echo Hello world Output :Hello world Ex2:**

\$x=10

\$echo "The value of x is \$x"

Entering these two commands will output the following text:

Output :The value of x is 10.

Ex 3:

\$echo -e "Here\bthe\bspaces\bare\bbackspaced"

Outputs the following text:

Output :Herthspacearbackspaced //one character before backslash is deleted. Ex 4:

\$echo -e "Unix \n is \n a \n os"

Output :\$Unix is a Os //prints in a new line Ex 5:

\$echo -e "unix \t is \t a \t os"

Output :unix is a os /\t –prints after 8 spaces Ex 6:

\$echo -e "unix \r is a os"

Output : is a os //moving cursor to beginning of line. //words before \r is deleted. Ex 7:

\$echo -e "unix \c is a os"

Output :unix // cursor in same line.

//words after \c deleted. Ex 8:

\$echo \$SHELL

Output :/bin/bash //prints the shell name

Printf Command-alternative to echo command

\$printf "No file entered" Output :No file entered

\$printf My current shell is %s\n" \$SHELL

Output :My current shell is /bin/bash //Here %s act as a placeholder for the value \$SHELL

%s-String **%f**-Floating point number

%d-Decimal integer **%x**-Hexadecimal integer

%o-octal integer

COMBINING COMMANDS

UNIX allows you to specify more than one command in the command line. Each command has to be separated from the other by a ; (semicolon).

Example

\$wc note; ls -l note

tThe above command first it displays the line count, word cont and byte or character count along with this it also display the details of note file.

When you learn to redirect the output of these commands you may even like to group them together within parentheses .

Example :

\$(wc note ; ls -l note) > newlist

The combined output of the two commands is now sent to the file newlist. Whitespace is provided here only for better readability. You might reduce a few keystrokes like this

\$wc note;ls -l note)>newlist

When a command line contains a semicolon, the shell understands that the command on each side of it needs to be processed separately. The ; here is known as a metacharacter, and you'll come across several metacharacters that have special meaning to the shell.

A command line can overflow or be split into multiple lines

A command is often keyed in. though the terminal width is restricted to 80 characters, that doesn't prevent you from entering a command, or a sequence of them, in one line even though the total width may exceed 80 characters. The command simply overflows to the next line though it is still in a single logical line.

Sometimes, you'll find it necessary or desirable to split a long command line into multiple lines. In that case, the shell issues a **secondary prompt**, usually `>`, to indicate to you that the command line isn't complete. This is easily shown with the echo command:

```
$echo "this is  
> a three-line  
>text message"
```

Output: this is a three-line text message

The `bc` command in Unix (short for Basic Calculator) is a command-line calculator that supports arbitrary precision arithmetic and allows scripting for complex calculations.

bc Command

Simply type `bc` in the terminal and start entering expressions:

```
$ bc  
5 + 3  
8
```

Exit with `Ctrl+D` or type `quit`.

Script Command

The `script` command in Unix is used to **record a terminal session**. It captures everything displayed on the terminal and saves it to a file, which can be useful for logging, debugging, or documentation purposes.

To start recording a session:

```
$ script
```

- This starts recording everything you type and see on the terminal.
- To stop recording, type `exit` or press **Ctrl+D**.

The output is saved in a file named `typescript` (default).

Save Output to a Specific File

```
$ script my_session.log
```

This records the terminal session into `my_session.log`.

uname Command in Unix

The `uname` command in Unix is used to display system information such as the operating system, kernel version, and system architecture.

```
$ uname
```

Default behavior, shows only the operating system name i e Linux

Display All System Information

```
$ uname -a
```

This provides:

- **Kernel name** (Linux)
- **Hostname** (myhostname)
- **Kernel version** (5.15.0-91-generic)
- **Build info** (#101-Ubuntu SMP)

- **Machine hardware name** (x86_64)
- **OS type** (GNU/Linux)

tty command

The tty command in Ubuntu (and other Unix-like systems) is used to display the file name of the terminal connected to the standard input.

Syntax

tty [options]

Common Uses

1. Check the current terminal device

\$ tty

Example output:

/dev/pts/0

This indicates that the user is working on a pseudo-terminal (pts/0), which is typical for SSH and terminal emulators.

2. Check if the input is a terminal

\$tty -s

This returns:

- **Exit code 0** (success) if the standard input is from a terminal.
- **Exit code 1** if the input is from a pipe or file.

3. Use tty in a script to check for terminal input

if tty -s;

then echo "Running in a terminal."

else echo "Not running in a terminal."

fi

stty

The stty (set terminal type) command in Ubuntu is used to change and print terminal line settings. It allows users to configure input and output settings of the terminal, such as baud rate, character size, control characters, and various other terminal behavior settings.

Basic Syntax:

\$stty [options]

Common Uses of stty

1. Display current terminal settings

\$stty -a

This lists all terminal settings.

Turn Off Terminal Echo (Useful for Password Inputs)

\$stty -echo

To turn it back on:

\$stty echo

Set Terminal Baud Rate(Baud rate in Ubuntu refers to the speed of data transmission in bits per second (bps) over a serial communication interface.)

Set the baud rate to 9600:\$ stty speed 9600

MEANING OF INTERNAL AND EXTERNAL COMMANDS

UNIX commands are classified into two types Internal Commands - Ex: echo

External Commands - Ex: ls, cat

Internal Command:

Internal commands are something which is built into the shell. For the shell built in commands, the execution speed is really high. It is because no process needs to be spawned for executing it.

For example, when using the "cd" command, no process is created. The current directory simply gets changed on executing it.

External Command:

External commands are not built into the shell. These are executable present in a separate file. When an external command has to be executed, a new process has to be spawned and the command gets executed.

For example, when you execute the "cat" command, which usually is at /usr/bin, the executable /usr/bin/cat gets executed.

```
$ type cd
```

```
cd is a shell builtin
```

```
$ type cat
```

```
cat is /bin/cat
```

For the internal commands, the type command will clearly say its shell built-in, however for the external commands, it gives the path of the command from where it is executed.

THE TYPE COMMAND: knowing the type of a command and locating it.

type - Display information about command type. The **type** command is a shell built-in that displays the kind of command the shell will execute, given a particular command name. It works like this: type *command* where "command" is the name of the command you want to examine. Here are some examples:

```
$type type
```

```
Output: type is a shell built-in
```

```
$type ls
```

```
Output: ls is aliased to 'ls --color=tty'
```

```
$type cp
```

```
Output: cp is /bin/cp
```

Here we see the results for three different commands. Notice that the one for ls (taken from a Fedora system) and how the ls command is actually an alias for the ls command with the "--color=tty" option added. Now we know why the output from ls is displayed in color!

THE ROOT LOGIN

Root: the system administrator's login. The unix system provides a special login name for the exclusive use of the administrator, it is called **root**. This account doesn't need to be separately created but comes with every system. Its password is generally set at the time of installation of the system and has to be used on logging in

Login: **root**

Password: *****

The prompt of the root is # other users(non privileged user) either \$ or %

Once you login as root, you are placed in root's home directory. Depending on the system, this could be / or **/root**

BECOMING THE SUPER USER: SU COMMAND

Any user can acquire super user status with the **su** command if she knows the root password.

Example, the user SDMIT becomes a super user in this way

```
$su Password:***** #pwd
```

```
/home/SDMIT
```

Though the current directory does not change the # prompt indicates the SDMIT now has powers of a super user. To be in root's home directory on superuser login, use **su -l**.

Creating a user's Environment: users often rush to the administrator with the complaint that a program has stopped running. The administrator first tries running it in a simulated environment. **su**, when used with a -, recreates the user's environment without taking the login password route:

```
$su - SDMIT
```

This sequence executes SDMIT's .profile and temporarily creates SDMIT's environment.

Su runs a separate sub shell, so this mode is terminated by hitting [Ctrl-d] or using **exit**.

UNIX FILES

NAMING FILES

UNIX permits file names to use most characters, but avoid spaces, tabs and characters that have a special meaning to the shell, such as:

& ; () | ? \ ' " ` [] { } < > \$ - ! /

It is recommended that only the following characters be used in filenames.

Alphabetic characters and numerals

The period(.), the hyphen(-) and underscore(_)

Case Sensitivity: uppercase and lowercase are not the same! These are three different files:

NOVEMBER November november

Length: can be up to 256 characters

Extensions: may be used to identify types of files libc.a - *archive, library file*

program.c - *C language source file* alpha2.f - *Fortran source file* xwd2ps.o - *Object/executable code* mygames.Z - *Compressed file*

BASIC FILE TYPES/CATEGORIES

File is a collection of records. So, files are divided into three categories

Ordinary file

Directory file

Device file

The UNIX file system contains several different types of files:

Ordinary Files or regular files

It contains only data as a stream of characters. An ordinary file itself divided into 2 types

Text file: contains only printable characters and you can often view the contents and make sense out of them. All C and Java files are example of text file. A text file contains lines of

characters where every line is terminated with the newline character, also known as **linefeed** (LF) when you press Enter while inserting text, the LF character is appended to every line. You won't see this character normally, but there is command (od) which can make it visible. **Binary file:** it contains both printable and unprintable characters that cover the entire ASCII range(0 to 255).most UNIX commands are example of binary files.

Directory files

Contains no data, but keeps some details of the files and subdirectories that it contains.

A directory file contains an entry for every file and sub directory that it houses. Each entry has two components

The filename

A unique Identification number for the file or directory(called the inode number)

A directory contains the filename but not the contents of file.

When you create or remove a file the kernel automatically updates its corresponding directory by adding or removing the enter i.e inode number associated with that file.

Device files

Used to represent a real physical device such as a printer, tape drive or terminal, used for Input/Output (I/O) operations

Unix considers any device attached to the system to be a file - including your terminal:

By default, a command treats your terminal as the standard input file (stdin) from which to read its input

Your terminal is also treated as the standard output file (stdout) to which a command's output is sent.

stdin and stdout will be discussed in more detail later

Two types of I/O: character and block

Usually only found under directories named /dev.

PARENT CHILD RELATIONSHIP/ UNIX FILE SYSTEM

All files in UNIX are related to one another. The file system in unix is a collection of all ordinary, directory and device files and organized in a hierarchical structure as shown in below figure.

The implicit feature of every UNIX file system is that there is a top which serves as reference point for all files. This top is called **root** & is represented by a /(front slash). Root is actually a directory. The root directory has a number of sub directories under it. These sub directories in turn have more sub directories and others files under them.

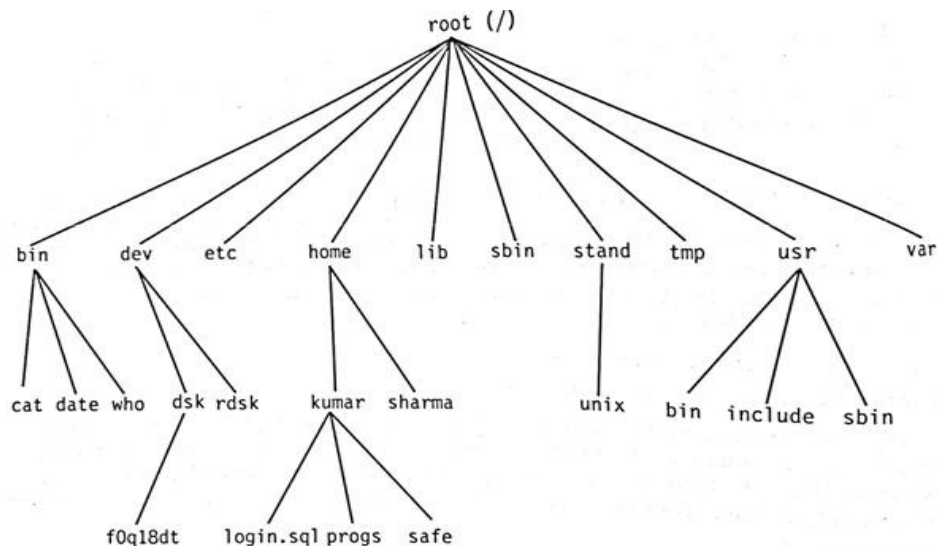


Figure 1.14.1 Unix file system

For instance bin and usr are two directories directly under root, while a second bin and kumar are sub directories under usr. Every file apart from root must have a parent. Thus the home directory is the parent of kumar, while / is the parent of home and grandparent of kumar. If you create a file login.sql under the kumar directory, then kumar will be the parent of this file.

The first group contains the files that are made available during system installation

/bin and /usr/bin: these are the directories where all the commonly used UNIX commands are found.

/sbin and /usr/sbin: If there's a command that you can't execute but the system administrator can execute, it would be probably in one of these directories.

/etc: this directory contains the configuration files of the system. You can change a very important aspect of system functioning by editing a text file in this directory. Your login name and password are stored in files /etc/passwd and etc/shadow

/dev: This directory contains all device files. These files don't occupy space on disk. there could be more sub directories like pts, dsk and rdsd in this directory

/lib and /usr/lib: Contains shared library files and sometimes other kernel-related files.

/usr and /include: contains the standard header files used by C programs. The statement `#include<stdio.h>` used in most C programs refers to the file stdio.h in this directory.

/usr/share/man: this is where the man pages are stored. There are separate subdirectories here (like man1, man2 etc) that contains the pages for each section. For instance, the man page of **ls** can be found in /usr/share/man/man1

User also work with their own files, they write programs, send and receive mail and also create temporary files. These files are available in the second group shown below

/tmp: the directory where users are allowed to create temporary files. These files are wiped away regularly by the system

/var: The variable part of the file system which contains all your print jobs and your outgoing and incoming mail.

/home: On many systems users are housed here. Kumar would have his home directory in /home/kumar.

THE HOME DIRECTORY AND THE HOME VARIABLE

HOME DIRECTORY : When log on to the system, UNIX automatically places you in a directory called the **home directory**.

It is created by the system when user account is opened.

If you log in using the login name sharma , you will land up in a directory that could have the pathname

/home/sharma

The shell variable HOME known's yours home directory

\$echo \$HOME

/home/sharma

You will be doing much of your work in your home directory and subdirectories.

Home variable: it is also called environment variables. **Environment variables** are a set of dynamic named values that can affect the way running processes will behave on a computer.

Here **\$HOME** is a environment variable it indicates the home directory of the current user: the default argument for the cd built-in command.

REACHING REQUIRED FILES- THE PATH VARIABLE

Path Variable:

The PATH environment variable is a colon-delimited list of directories that your **shell** searches through when you enter a command.

Program files (executables) are kept in many different places on the **Unix** system. Your **path** tells the Unix shell where to look on the system when you request a particular program.

To find out what your path is, at the Unix shell prompt echo \$PATH Your path will look something like the following.

/usr2/username/bin:/usr/local/bin:/usr/bin:

You will see your username in place of username. Using the above example path, if you enter the ls command, your shell will look for the appropriate executable file in the following order: first, it would look through the directory /usr2/username/bin, then /usr/local/bin, then /usr/bin, and finally the local directory, indicated by the . (a period).

RELATIVE AND ABSOLUTE PATHNAMES.

Relative Pathnames

Pathnames that doesn't begin with /, specify locations relative to your current working directory.

Uses either the current or parent directory as reference and specifies path relative to it. A relative pathname uses one of these cryptic symbols.

.(a single dot)-this represents the current directory.

.. (two dots) this represents the parent directory

Command Function

cd Returns you to your login directory

cd ~ Also returns you to your login directory

cd / Takes you to the entire system's root directory

cd /root Takes you to the home directory of the root or superuser,account created at installation, you must be root user to access this directory.

`cd /home` Takes you to the home directory where user login directories are usually stored

`cd ..` Moves you up one directory

`cd ~otheruser` Takes you to the otheruser's login directory

`cd /dir/subdirfoo` Regardless of which directory you are in, the absolute path takes you directly to `subdirfoo`, a subdirectory of `dir`.

Ex 1: Assume the current directory is `/home/kumar/progs/data/text`, using `cd ..` will move one level up

\$pwd

/home/kumar/progs/data/text

\$ cd ..

\$pwd

/home/kumar/progs/data

Ex 2 : To move two levels up

\$pwd

/home/kumar/progs

\$ cd ../../

\$pwd

/home

Ex 3: My present location is `/etc/samba` and now I want to change directory to `/etc`. Using relative path: **\$ cd ..**

Using absolute path: **\$cd /etc**

Ex 4: My present location is `/var/ftp/` and I want to change the location to `/var/log` Using relative path: **cd ../log**

Using absolute path: **cd /var/log**

Ex 5: My present location is `/etc/lvm` and I want to change my location to `/opt/oradba` Using relative path: **cd ../../opt/oradba**

Using absolute path: **cd /opt/oradba**

Absolute Pathnames:

If the first character of a pathname is `/` the files location must be determined with respect to root(`/`) . Such a pathname is called absolute pathname.

`cat /home/kumar`

When you have more than one `/` in a pathname for such `/` you have to descend one level in the file system. Thus Kumar is one level below home and two levels below root.

When you specify a file y using frontslashes to demarcate the various levels,you have a mechanism of identifying a file uniquely.No two files in a UNIX system can have same absolute pathnames.

When you specify the date command, the system has to locate the file date from a list of directories specified in the `PATH` variable and then execute it.

However if you know the location of a command in prior, for example date is usually located in `/bin` or `/usr/bin` . Use absolute path name i,e precede its name with complete path

\$/bin/date

For example if you need to execute program less residing in `/usr/local/bin` you need to enter the absolute path name

\$/usr/local/bin/less

DIRECTORY COMMANDS – PWD, CD, MKDIR, RMDIR COMMANDS

pwd (print working directory) (checking your current directory)

As the name states, command '**pwd**' prints the current working directory or simply the directory user is, at present. It prints the current directory name with the complete path starting from root (/). This command is built in shell command and is available on most of the shell – bash, Bourne shell, ksh,zsh, etc.

Basic syntax

\$pwd [option]

Options Description

-L (logical) Use PWD from environment, even if it contains symbolic links

-P (physical) Avoid all symbolic links

–help Display this help and exit

–version Output version information and exit

If both '**-L**' and '**-P**' options are used, option '**L**' is taken into priority. If no option is specified at the prompt, pwd will avoid all symlinks, i.e., take option '**-P**' into account.

cd: Changing The Current Directory

The **cd** command, which stands for "change directory", changes the shell's current working directory.

The **cd** command is one of the commands you will use the most at the command line in UNIX.

It allows you to change your working directory. You use it to move around within the hierarchy of your file system.

Ex 1 When used with an argument it changes the current directory to the directory specified as argument for example assume sdmit is a directory under user directory Kumar. To change from Kumar directory to sdmit directory , issue the command as follows

\$pwd

/home/kumar

\$cd sdmit

\$pwd

/home/kumar/sdmit

When cd used without arguments: cd when used without arguments reverts to home directory

\$pwd

/home/kumar/sdmit

\$cd

cd without argument will change directory from sdmit to its home directory Kumar

\$pwd/home/kumar

mkdir: "making directory".

mkdir is used to create directories on a file system.

If the specified *DIRECTORY* does not already exist,**mkdir** creates it. More than one

□

□

DIRECTORY may be specified when calling **mkdir**.

mkdir syntax

mkdir [OPTION ...] DIRECTORY ...

Ex 1: To create a directory named sdmit issue the following command.

\$mkdir sdmit

sdmit directory is created under present working directory.

Assume that pwd is /home/kumar , then sdmit directory is created under kumar directory.

Ex .2: To create three directories at a time, named patch, dbs, doc, pass directory names as arguments.

\$mkdir patch dbs doc

Ex 3: To create a directory tree:

To create a directory named gmit and create two subdirectories named cse and ise under sdmit, issue the command. sdmit is a parent directory.

\$mkdir parent directory sub-directories

\$mkdir sdmit sdmit/cse sdmit/ise

Ex 4: Error while creating a directory tree

\$mkdir sdmit/cse sdmit/ise

mkdir: Failed to make a directory “sdmit/cse”; no such file or directory mkdir: Failed to make a directory “sdmit/ise”; no such file or directory

Error is due to the fact that the parent directory named sdmit is not created before creating sub directories cse and ise.

Ex 5: **\$mkdir test**

mkdir: Failed to make directory “test”; Permission denied. This can happen due to:

The directory named test may already exist

There may be an ordinary file by the same name in the current directory.

The permissions set for the current directory do not permit the creation of files and directories by the user.

rmdir: Removing Directories

The **rmdir** utility removes the directory entry specified by each directory argument, provided the directory is empty.

Ex 1: **\$rmdir progs** removes the directory named progs

Arguments are processed in the order given. To remove both a parent directory and a subdirectory of that parent, the subdirectory must be specified first, so the parent directory is empty when **rmdir** tries to remove it. The reverse logic of mkdir is applied.

\$rmdir subdirectories parent directory

\$rmdir sdmit/cse sdmit/ise sdmit

You can't delete a directory with rmdir unless it is empty. In this example sdmit directory cannot be removed until the sub directories cse and ise are removed.

You can't remove a sub directory unless you are place in a directory which is hierarchically above the one you have chosen to remove.

THE DOT (.) AND DOUBLE DOTS (..) NOTATIONS TO REPRESENT PRESENT AND PARENT DIRECTORIES AND THEIR USAGE IN RELATIVE PATH NAMES.

A relative pathname uses one of these cryptic symbols.

. (a single dot)-this represents the current directory.

.. (two dots) ☐ this represents the parent directory cd .. Moves you up one directory

\$pwd

/home/kumar/progs

\$ cd ../..

\$pwd

/home